

Stream in C++

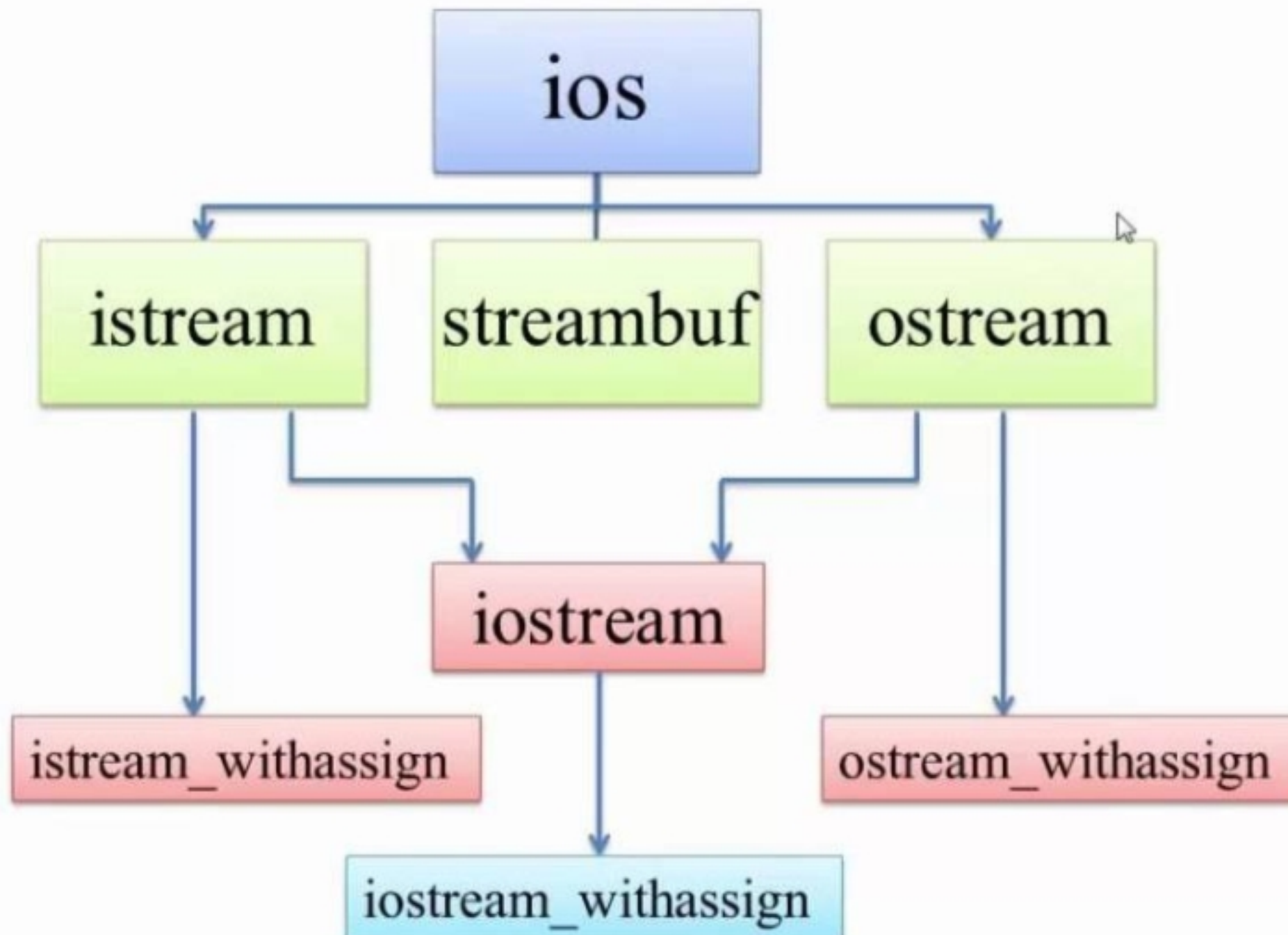
Stream

- The iostream library is an object-oriented library that provides input and output functionality using streams.
- A stream is an abstraction that represents a device on which input and output operations are performed. A stream can basically be represented as a source or destination of characters of indefinite length.
- Streams are generally associated to a physical source or destination of characters, like a disk file, the keyboard, or the console, so the characters gotten or written to/from our abstraction called stream are physically input/output to the physical device.

Stream Classes

- Stream classes in C++ are used to input and output operations on files and io devices. These classes have specific features and to handle input and output of the program.
- It provides **cin** and **cout** Objects for reading from input and writing to output respectively.
- All these classes are defined in the file `iostream` and `fstream` header file.

Class Hierarchy



Class Hierarchy

- **ios** class is topmost class in the stream classes hierarchy. It is the base class for **istream**, **ostream**, and **streambuf** class.
- **istream** and **ostream** serves the base classes for **iostream** class. The class **istream** is used for input and **ostream** for the output.
- Class **ios** is indirectly inherited to **iostream** class using **istream** and **ostream**. To avoid the duplicity of data and member functions of ios class, it is declared as virtual base class when inheriting in **istream** and **ostream**.
- The **_withassign** classes are provided with extra functionality for the assignment operations.

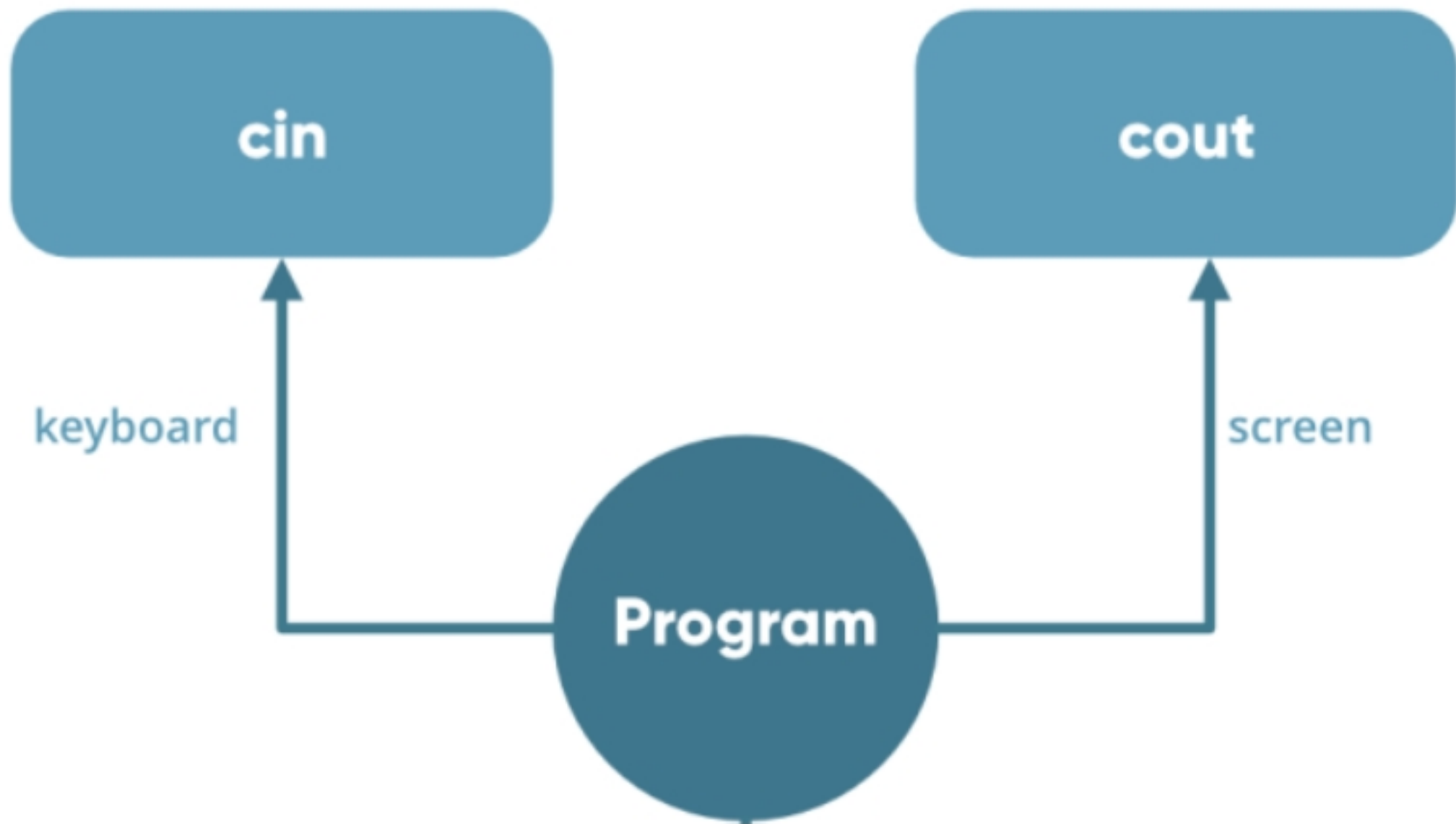
Stream Classes

- **ios class:** The ios class is responsible for providing all input and output facilities to all other stream classes.
- **istream class:** This class is responsible for handling input stream. It provides number of function for handling chars, strings and objects such as **get**, **getline**, **read**, **ignore**, **putback** etc..
- **ostream class:** This class is responsible for handling output stream. It provides number of function for handling chars, strings and objects such as **write**, **put** etc..
- **iostream class:** This class is responsible for handling both input and output stream as both **istream** class and **ostream** class is inherited into it. It provides function of both **istream** class and **ostream** class for handling chars, strings and objects such as **get**, **getline**, **read**, **ignore**, **putback**, **put**, **write** etc..
- **streambuf class:** Provides an interface to physical devices through buffer. Acts as a base for filebuf using **ios** files.

Basic Input Output Operation

- C++ comes with libraries that provide us with many ways for performing input and output. In C++ input and output are performed in the form of a sequence of bytes or more commonly known as streams.
- Input Stream: If the direction of flow of bytes is from the device(for example, Keyboard) to the main memory then this process is called input.
- Output Stream: If the direction of flow of bytes is opposite, i.e. from main memory to device(display screen) then this process is called output.

Input Output Operation



Header files available in C++ for Input/Output operations

- **iostream**: iostream stands for standard input-output stream. This header file contains definitions to objects like cin, cout, cerr etc.
- **iomanip**: iomanip stands for input output manipulators. The methods declared in this files are used for manipulating streams. This file contains definitions of setw, setprecision, etc.
- **fstream**: This header file mainly describes the file stream. This header file is used to handle the data being read from a file as input or data being written into the file as output.

Cin and Cout

- Standard output stream (**cout**): Usually the standard output device is the display screen. **cout** statement is the instance of the ostream class. The data needed to be displayed on the screen is inserted in the standard output stream (cout) using the insertion operator(<<).
- Standard input stream (**cin**): Usually the input device in a computer is the keyboard. **cin** statement is the instance of the class istream. The extraction operator(>>) is used along with the object **cin** for reading inputs. The extraction operator extracts the data from the object cin which is entered using the keyboard.

Get and put function

- the class `istream` and `ostream` define two member functions `get()`, and `put()` to handle the single character input/output operations.
- Syntax:
`cin.get(char_Var);`
`cout.put(char_Var);`

Example:

```
#include <iostream>
using namespace std;
int main()
{
    int count=0;
    char ch;
    cout<<"INPUT TEXT \n";
    cin.get( ch );
    while ( ch !='\n' )
    {
        cout.put( ch);
        count++;
        cin.get( ch );
    }
    cout<< "\n Number of characters ="<<count<<"\n";
    return 0;
}
```

Getline and write functions

- You can read and display lines of text more efficiently using the line oriented input/output functions. They are:
 - `getline()`
 - `write()`
- The `getline()` function reads the entire line of texts that ends with a newline character. The general form of `getline()` is:
 - `cin.getline (line, size);`
- The `write()` function displays the entire line of text, and the general form of writing this function is:
 - `cout.write (line, size);`

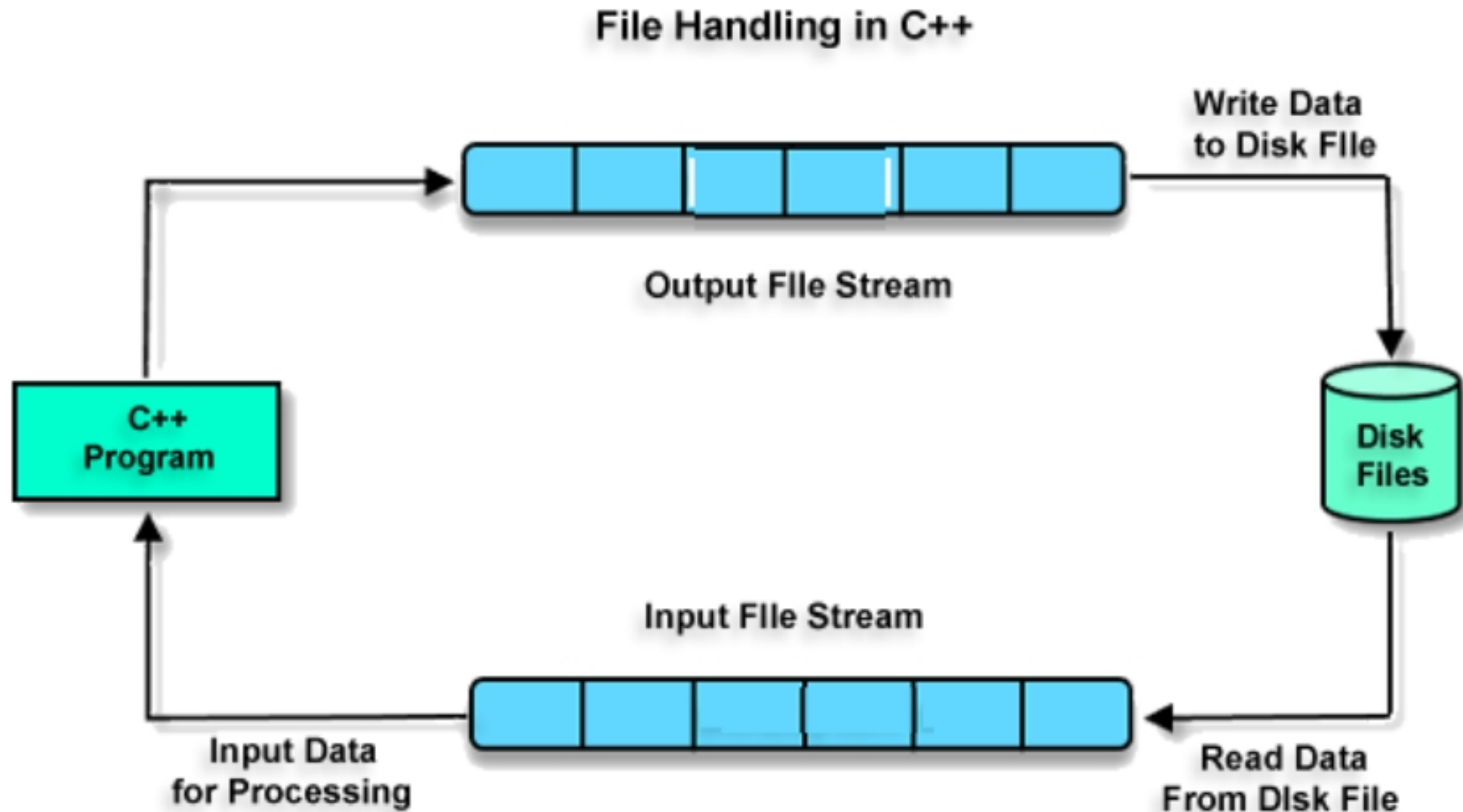
Example:

```
//Reading Strings With getline()
#include <iostream>
using namespace std;
int main()
{
    int size=20;
    char city[20];
    cout<<"enter city name:\n ";
    cin>>city;
    cin.ignore();
    cout<<"\ncity name:"<<city<<"\n\n";
    cout<<"enter city name again: \n";
    cin.ignore();
    cin.getline(city,size);
    cout<<"city name now:";
    cout.write(city,20);
    cout<<"enter another city name: \n";
    cin.getline(city,size);
    cout <<"New city name:"<<city<<"\n\n";
    return 0;
}
```

File Stream

- When a large amount of data is to be handled in such situations hard disk or pen drive are needed to store the data . A file is a collection of related data stored in a particular area on a disk.
- Programs can be designed to perform the read and write operations on these files .
- The I/O system of C++ handles file operations which are very much similar to the console input and output operations .It uses file streams as an interface between the programs and files. The stream that supplies data to the program is called input stream and the one that receives data from the program is called output stream.
- The input operation involves the creation of an input stream and linking it with the program and input file. Similarly, the output operation involves establishing an output stream with the necessary links with the program and output file.

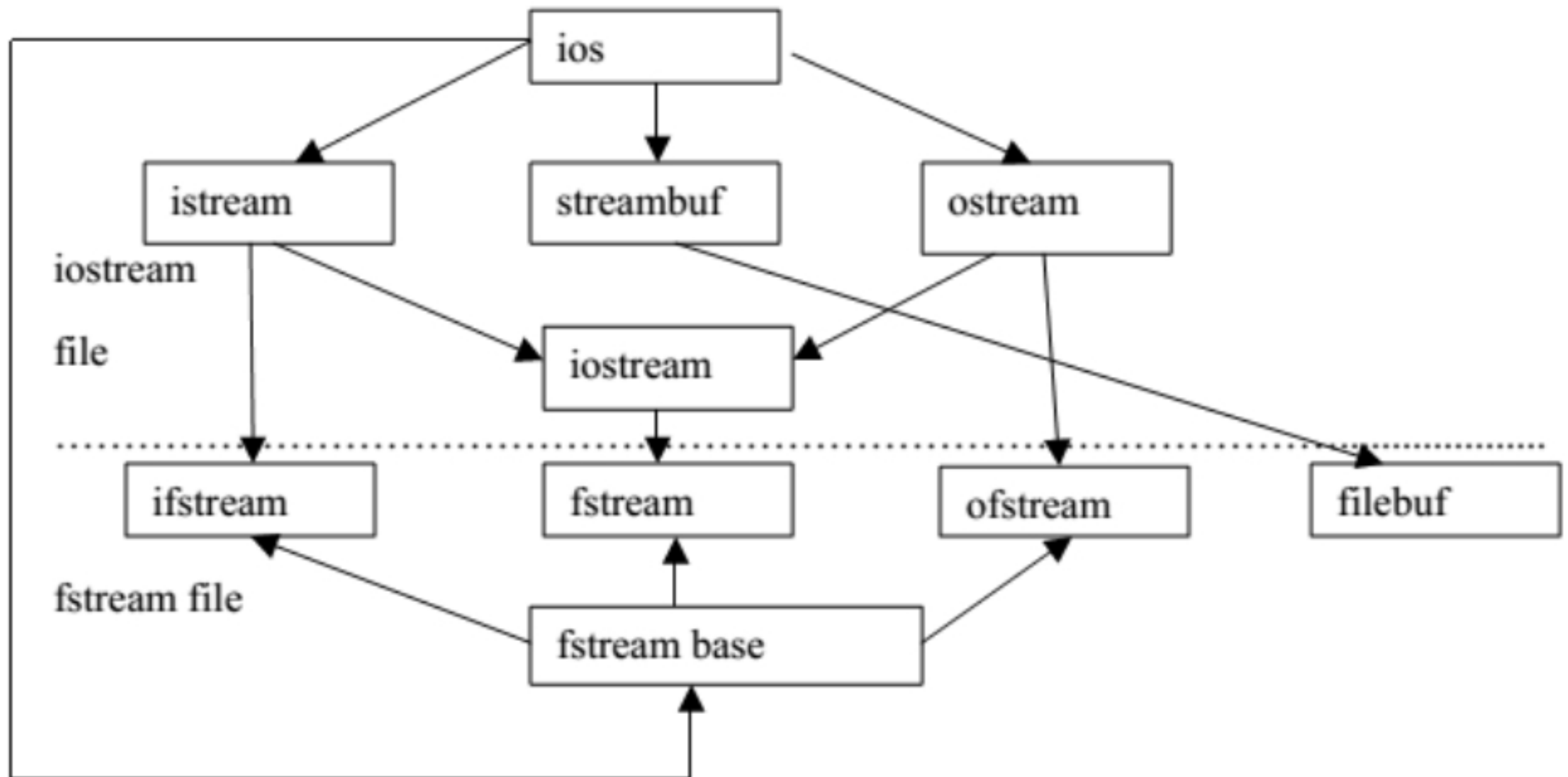
Input and Output Stream



File Stream Classes

- The I/O system of C++ contains a set of classes that defines the file handling methods. These include **ifstream**, **ofstream** and **fstream**. These classes are derived from **fstreambase** and form the corresponding **iostream** class. These classes ,designed to manage the disk files.

File Stream Classes



File Stream Classes

Class	Contents
filebuf	Its purpose is to set the file buffers to read and write. Contains Openprot constant used in the open() of file stream classes. Also contain close() and open() as members.
fstreambase	Provides operations common to file streams. Serves as a base for fstream , ifstream and ofstream class. Contains open() and close() functions.
ifstream	Provides input operations. Contains open() with default input mode. Inherits the functions get() , getline() , read() , seekg() , tellg() functions from istream .
ofstream	Provides output operations. Contains open() with default output mode. Inherits put() , seekp() , tellp() and write() functions from ostream .
fstream	Provides support for simultaneous input and output operations. Contains open with default input mode. Inherits all the functions from istream and ostream classes through iostream .

Steps of File Operations

- For using a disk file the following things are necessary
 - Suitable name of file
 - Data type and structure
 - Purpose
 - Opening Method
- The filename is a string of characters that makeup a valid filename for the operating system. It may contain two parts ,primary name and optional period with extension.
 - Examples: Input.data, Test.doc etc.
- For opening a file firstly a file stream is created and then it is linked to the filename. A file stream can be defined using the classes ifstream, ofstream and fstream that contained in the header file fstream. The class to be used depends upon the purpose whether the write data or read data operation is to be performed on the file.

Opening a File

- A file can be opened in two ways:
 - Using the constructor function of class.
 - Using the member function `open()` of the class.
- The first method is useful only when one file is used in the stream. The second method is used when multiple files are to be managed using one stream.
- Default File Opening Modes :
 - `ifstream` `ios::in`
 - `ofstream` `ios::out`
 - `fstream` `ios::in | ios::out`

File Opening Modes

Member Constant	Stands For	Access
<code>ios::in</code>	Input	File open for reading: the internal stream buffer supports input operations.
<code>ios::out</code>	output	File open for writing: the internal stream buffer supports output operations.
<code>ios::binary</code>	binary	Operations are performed in binary mode rather than text.
<code>ios::ate</code>	at end	The output position starts at the end of the file.
<code>ios::app</code>	append	All output operations happen at the end of the file, appending to its existing contents.
<code>ios::trunc</code>	truncate	Any contents that existed in the file before it is open are discarded.
<code>ios::nocreate</code>	No create	open fails if the file does not exist
<code>ios::noreplace</code>	No replace	open fails if the file already exist

Using the constructor function of class

- While using constructor for opening files, filename is used to initialize the file stream object.
- This involves the following steps:
 - Create a file stream object to manage the stream using the appropriate class i.e the class ofstream is used to create the output stream and the class ifstream to create the input stream.
 - Initialize the file object using desired file name.
 - For example:

```
ofstream outFile("results");    //output only
ifstream inFile ("data");        //input only
```

Closing a File:

Syntax: FileObject.close();

Exmple: outFile.close();
 inFile.close();

Using the member function open() of the class

- The function open() can be used to open multiple files that uses the same stream object. For example to process a set of files sequentially, in such case a single stream object can be created and can be used to open each file in turn.
- This can be done as follows:
 - `File_stream_class stream_object;`
`Stream_object.open("filename", mode)`
`ofstream outFile;`
`outFile.open("sample.txt");//Default ios::out mode`

`ifstream inFile;`
`inFile.open("sample.txt");//Default ios::in mode`
- All these flags can be combined using the bitwise operator OR (|). For example, if we want to open the file "example.bin" in binary mode to add data we could do it by the following call to member function open():
`fstream file;`
`file.open ("example.bin", ios::out | ios::app | ios::binary);`

Input output one character

```
#include<iostream>
#include<fstream>
using namespace std;
int main()
{
    char ch;
    ofstream fout("kcc.txt");
    cout<<"Enter a Line of text:";
    do
    {
        cin.get(ch);
        fout.put(ch);
    }while(ch!='\n');
    fout.close();
    ifstream fin("kcc.txt");
    cout<<"Data From Disk:\n";
    fin.get(ch);
    while(!fin.eof())
    {
        fin.get(ch);
        cout<<ch;
    }
    fin.close();
    cout<<"\nData Read / Write successfully...\n\n\n";
    return 0;
}
```

Example

```
#include<iostream>
#include<fstream>
using namespace std;
int main()
{
    ofstream fout;
    fout.open("country");
    fout<<"United states of America \n";
    fout<<"United Kingdom\n";
    fout<<"South korea\n";
    fout.close();
    fout.open("capital");
    fout<<"Washington\n";
    fout<<"London\n";
    fout<<"Seoul \n";
    fout.close();
    const int N =80;
    char line[N];
    ifstream fin;
    fin.open("country");
```

```
    cout<<"contents of country file: \n";
    while (fin)
    {
        fin.getline(line,N);
        cout<<line<<endl;
    }
    fin.close();
    fin.open("capital");
    cout<<"contents of capital file:\n";
    while(fin)
    {
        fin.getline(line,N);
        cout<<line<<endl;
    }
    fin.close();
    return 0;
}
```

Example of fstream

```
#include <iostream>
#include <fstream>

using namespace std;

int main()
{
    fstream file;
    file.open("E:\kcc.txt",ios::out);
    if(!file)
    {
        cout<<"File creation failed";
        system("pause");
        exit(0);
    }
    cout<<"New file created:\n";
    file<<"Kntipur City College";

    file.close();
    cout<<"DATA From file:\n";
    file.open("E:\kcc.txt",ios::in);
    char ch;
    while (!file.eof())
    {
        file >>ch;
        cout << ch;
    }
    file.close();
    return 0;
}
```

File Pointers and Manipulators

- Each file has two pointers known as file pointers, one is called the input pointer and the other is called output pointer. The input pointer is used for reading the contents of a given file location and the output pointer is used for writing to a given file location.
- **Default file pointer position**
- When a file is opened in read-only mode, the input pointer is automatically set at the beginning so that file can be read from the start.
- Similarly when a file is opened in write-only mode the existing contents are deleted and the output pointer is set at the beginning.
- In case an existing file is to be opened in order to add more data, the file is opened in 'append' mode. This moves the pointer to the end of file.

Functions for Manipulations of File pointer

- `seekg()` Moves get pointer (input) to a specified location.
- `seekp()` Moves put pointer (output) to a specified location.
- `tellg()` Give the current position of the get pointer.
- `tellp()` Give the current position of the put pointer.
- **Consider the following statements:**
`ofstream fileout;`
`fileout.open("hello",ios::app);`
`int p=fileout.tellp();`
- On execution of these statements, the output pointer is moved to the end of file "hello" And the value of p will represent the number of bytes in the file.

Seek Function

- 'Seek' functions `seekg()` and `seekp()` can also be used with two arguments as follows:
- `File_object.seekg (offset,refposition);`
- `File_object.seekp (offset,refposition);`
- The parameter `offset` represents the number of bytes the file pointer is to be moved from the location specified by the parameter `refposition`.
- The `refposition` takes one of the following three constants defined in the `ios` class:
 - `ios::beg` Start of file
 - `ios::cur` Current position of the pointer
 - `ios::end` End of file
- The `seekg()` function moves the associated file's '**get**' pointer while the `seekp()` function moves the associated file's '**put**' pointer.

Examples

- Pointer offset calls Seek call Action
- `file.seekg(o,ios::beg)` Go to start
- `file.seekg(o,ios::cur)` Stay at the current position
- `file.seekg(o,ios::end)` Go to the end of file
- `file.seekg(m,ios::beg)` Move to (m+1)th byte in the file
- `file.seekg(m,ios::cur)` Go forward by m byte from current position
- `file.seekg(-m,ios::cur)` Go backward by m bytes from current position.
- `file.seekg(-m,ios::end)` Go backward by m bytes from the end

Example

```
#include <fstream>
#include <iostream>
using namespace std;
int main ()
{
    char ch;
    fstream obj;
    obj.open("kcc.txt", ios::in|ios::out);
    obj<<"Kantipur City College";
    int pos=6;
    obj.seekg(pos);
    while(!obj.eof())
    {
        obj>>ch;
        cout<<ch;
    }
    obj.close();
    return 0;
}
```


Example

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    fstream file;
    file.open("kcc.txt",ios::in | ios::out);
    file<<"Kantipur City College";
    cout<< file.tellg() << endl;
    file.seekg(8, ios::beg);
    cout << file.tellg() << endl;
    char c = file.get();
    cout << c << endl;
    cout << file.tellg() << endl;
    file.seekg(10, ios::cur);
    cout << file.tellg() << endl;
    c = file.get();
    cout << c << endl;
    cout << file.tellg() << endl;
    file.seekp(7, ios::beg);
    file.put('Z');
    file.seekg(-7, ios::end);
    cout << "End:" << file.tellg() << endl;
    c = file.get();
    cout << c << endl;
    file.close();
    return 0;
}
```

Binary File

- Binary files are still flat files; however, they are in basic binary format rather than ASCII. Remember that everything in a computer is binary (1's and 0's). The binary format has each file stored in a byte-by-byte format. This produces some odd-looking results if we try to view binary files like plain text.
- Read and write function is used to for binary file IO. If we need to read and write objects then we can use read and write function. These functions are specially designed to work with large amount of data or group of data. Like array or objects.

Read and write function

- Syntax:

To write object's data members in a file :

// Here file_obj is an object of **ofstream**

```
file_obj.write((char *) & class_obj, sizeof(class_obj));
```

Example: fout.write((char*)&std,sizeof(std));

To read file's data members into an object :

// Here file_obj is an object of **ifstream**

```
file_obj.read((char *) & class_obj, sizeof(class_obj));
```

Example: fin.read((char*)&std,sizeof(std));

Example

```
#include <iostream>
#include <fstream>
using namespace std;
class student
{
    private:
        char name[30];
        int age;
    public:
        void getData(void)
        {
            cin.ignore();
            cout<<"Enter name:";
            cin.getline(name,30);
            cout<<"Enter age:";
            cin>>age;
        }
        void showData(void)
        {
            cout<<"Name: "<<name<<endl;
            cout<<"Age: "<<age<<endl;
        }
};
int main()
{
    student std;
```

```
    ofstream fout;
    int n;
    fout.open("aaa.txt",ios::out|ios::binary);
    cout<<"How many Record you want to add:";
    cin>>n;
    while(n>=1)
    {
        std.getData();
        fout.write((char*)&std,sizeof(std));
        n--;
    }
    fout.close();
    ifstream fin;
    fin.open("aaa.txt",ios::in);
    fin.read((char*)&std,sizeof(std));
    n=1;
    while(fin)
    {
        cout<<"Record "<<n++<<":\n";
        std.showData();
        fin.read((char*)&std,sizeof(std));
    }
    fout.close();
    return 0;
}
```